Object-Oriented Design Patterns

Sample study notes written for StudyBuddy. Free to share.

What a design pattern is

A design pattern is a named, reusable solution to a problem that keeps appearing in object-oriented design. A pattern is not finished code that can be copied; it is a description of classes, their roles and how they collaborate, which must be adapted to each situation.

Every pattern has four parts: a name, the problem it solves, the solution (the participating classes and their responsibilities) and its consequences, meaning the trade-offs of using it. Patterns give developers a shared vocabulary: saying "this is an Observer" explains a whole structure in one word.

Patterns are traditionally grouped into three families. Creational patterns deal with how objects are created. Structural patterns deal with how classes and objects are combined into larger structures. Behavioural patterns deal with how objects communicate and share responsibilities.

Strategy (behavioural)

Problem: a class needs one of several interchangeable algorithms, and the choice may change at run time. Putting every algorithm inside the class with an if or switch statement makes it large and forces an edit whenever an algorithm is added.

Solution: define a Strategy interface with one method for the algorithm. Each algorithm becomes a concrete strategy class. The context class holds a reference to a Strategy and delegates the work to it.

Example: a checkout calculates shipping cost. A ShippingStrategy interface declares cost(order), with StandardShipping, ExpressShipping and PickupInStore as implementations. The Checkout class is given a strategy and never needs to know which one it has. Adding overnight shipping means adding one class.

Consequences: new algorithms can be added without modifying the context, which is the open-closed principle in action, and each strategy can be tested separately. The cost is more classes, and clients must know enough to choose a strategy.

Observer (behavioural)

Problem: when one object changes, several other objects must react, but the changing object should not depend on all of them.

Solution: the subject keeps a list of observers that implement an Observer interface with an update method. Observers register themselves with the subject, and whenever the subject's state changes it notifies every registered observer.

Example: in a weather application, a WeatherStation is the subject. A CurrentConditionsDisplay, a StatisticsDisplay and an AlertService register as observers. When a new temperature arrives,

the station notifies all three, and each decides what to do with the reading.

Consequences: the subject and observers are loosely coupled, and observers can be added or removed at run time. Graphical user interfaces use this pattern heavily: a button notifies its listeners when it is clicked. A risk is that the order of notifications is not guaranteed, and an observer that is never unregistered can keep objects alive longer than intended.

Factory Method (creational)

Problem: a class must create objects, but the exact class to create should be decided elsewhere, or may change.

Solution: creation is moved into a factory method that returns an object through an interface. Subclasses, or a separate factory class, decide which concrete class to instantiate. Client code works only with the interface.

Example: a document editor creates pages. A PdfExporter and a WordExporter each implement createPage() to return the right kind of page, while the shared export logic in the base class calls createPage() without knowing which kind it gets.

Consequences: client code is no longer tied to concrete classes, and creation logic, such as validation or choosing a class from a type code, lives in one place.

Adapter (structural)

Problem: an existing class does what is needed but has the wrong interface, often because it comes from a third-party library that cannot be changed.

Solution: an adapter class implements the interface the client expects and translates each call into calls on the adapted object.

Example: an application expects a PaymentProcessor interface with a pay(amount) method. A payment provider's library offers a class with a method called submitCharge(cents, currencyCode). A ProviderPaymentAdapter implements PaymentProcessor and converts the amount to cents before calling submitCharge. Switching providers later means writing a new adapter, not changing the application.

Consequences: the rest of the system is protected from the third-party interface, which also makes it easy to replace the library with a fake in tests.

Facade (structural)

Problem: a subsystem has many classes, and clients that use it directly become coupled to all of them.

Solution: a facade class offers a small, simple interface for the common tasks and delegates to the subsystem's classes behind it.

Example: a home theatre has an amplifier, a projector, a streaming player and lights. A HomeTheatreFacade offers watchMovie(title) and endMovie(), and each method makes the right calls on the devices in the right order.

Consequences: clients become simpler and depend only on the facade. The subsystem can still be used directly when a client needs more control. Facade differs from Adapter in intent: an adapter converts one interface into another, while a facade simplifies a whole group of interfaces.

Builder (creational)

Problem: an object has many optional parts, and constructors with long parameter lists are hard to read and easy to call with arguments in the wrong order.

Solution: a separate builder object collects the parts step by step through named methods and creates the finished object at the end with a build method.

Example: an HTTP request has a URL, a method, headers, a body and a timeout. Code such as RequestBuilder.to(url).method("POST").header("Accept", "json").timeout(30).build() reads clearly and leaves out the parts that are not needed. The build method can check that the combination of parts is valid before creating the object.

Command (behavioural)

Problem: an action must be treated as an object so that it can be queued, logged, undone or chosen at run time.

Solution: a Command interface declares an execute method. Each action becomes a concrete command class that stores everything needed to perform it. An invoker runs commands without knowing what they do.

Example: a text editor turns each edit into a command object. Keeping executed commands on a stack makes undo possible: each command also implements undo, and the editor pops and undoes the last command.

Singleton (creational), used with care

Singleton ensures a class has only one instance and gives global access to it. It is easy to overuse: a singleton is a global variable in disguise, it hides dependencies and it makes unit testing harder because tests cannot easily replace it. Passing a shared object in through constructors, as described by the dependency inversion principle, is usually a better choice.

Choosing a pattern

Situation	Pattern
Several interchangeable algorithms	Strategy
Many objects must react to a change	Observer
Hide which concrete class is created	Factory Method
Use a class whose interface does not fit	Adapter
Simplify a complicated subsystem	Facade
Create an object with many optional parts	Builder
Treat actions as objects (undo, queues)	Command

A pattern should solve a problem the design actually has. Adding patterns where there is no problem only adds classes and makes the code harder to follow.